

DebtEase AI: AI-Powered Debt Relief & Financial Recovery Platform

Project Description:

DebtEase AI harnesses Generative AI to help borrowers manage debt, understand their financial health, and negotiate fair settlements with lenders. The platform includes a Financial Health Engine, which calculates EMI ratio, monthly surplus, and a debt stress score; a Settlement Prediction module, which recommends a realistic settlement amount with an AI-generated rationale; and an AI Negotiation Letter Generator, which drafts a professional, lender-specific negotiation letter tailored to the borrower's financial situation.

Utilizing Google's Gemini API, DebtEase AI processes borrower-submitted loan and income data to deliver fast, personalized financial guidance. Built with FastAPI and React.js and deployed via Docker, the platform ensures a secure, user-friendly experience. With JWT-based authentication, secure API key management via .env, and rule-based fallback logic when the AI service is unavailable, DebtEase AI empowers financially distressed borrowers to make informed decisions with confidence.

Scenarios

Scenario 1: A borrower with three overdue personal loans enters their outstanding balance, EMI, overdue duration, and monthly income. The system calculates a debt stress score, flags the highest-risk loan, and generates a recommended settlement amount with a plain-language explanation, all within seconds.

Scenario 2: A self-employed borrower with irregular income requests an AI-generated negotiation letter for their home loan lender. DebtEase AI drafts a professional, empathetic letter referencing the borrower's financial hardship and proposed settlement terms, ready to download or send.

Scenario 3: A borrower who previously submitted a settlement request returns to check its status. The dashboard shows their updated financial health score, a timeline of past negotiation letters, and guidance on next steps if the lender has not yet responded.

Technical Architecture:

The architecture diagram for this project (User Layer → Frontend Layer → API Layer → AI & Financial Processing Layer → Database Layer) is provided separately in the Solution Architecture document. Refer to that document for the full layered diagram.

Description: DebtEase AI utilizes a modular layered architecture to deliver rapid, AI-driven financial guidance. The frontend provides a responsive borrower dashboard built with React.js and Vite, while the FastAPI-based backend orchestrates authentication, data validation, and prompt construction. It interfaces with the Google Gemini API to generate tailored negotiation letters and strategies, with secure deployment managed via Docker.

Note: Since this project uses a database (SQLite via SQLAlchemy) to persist users, loans, settlements, and negotiation history, an ER Diagram is included in the accompanying Data Flow Diagram and database schema documentation.

Pre-requisites:

- **FastAPI Framework Knowledge:** [FastAPI Documentation](#)
- **Google Gemini API Familiarity:** [Gemini API Documentation](#)
- **React.js, HTML, and CSS Skills:** [React Documentation](#)
- **Python Programming Proficiency:** [Python Documentation](#)
- **Version Control with Git:** [Git Documentation](#)
- **Development Environment Setup:** [FastAPI Installation Guide](#)
- **Docker for Containerized Deployment:** [Docker Documentation](#)

Project Workflow:

Milestone 1: Model Selection and Architecture

- **Activity 1.1:** Generate a Google Gemini API key and configure it securely in the project environment.
- **Activity 1.2:** Research and select the appropriate generative AI model from Gemini for financial negotiation content generation.
- **Activity 1.3:** Define the architecture of the application, detailing interactions between the frontend, backend, and AI integration.
- **Activity 1.4:** Set up the development environment, installing necessary libraries and dependencies for FastAPI and the Gemini API.

Activity 1.1: Generate a Gemini API Key

Before the application can communicate with the Gemini LLM, you need a valid API key. Visit <https://aistudio.google.com> and sign in with a Google account. Navigate to “Get API Key,” create a new key, and copy it immediately — it will not be shown again.

Add the key to a .env file in the project root: `GEMINI_API_KEY=your_copied_api_key_here`. The app loads this automatically at startup using python-dotenv. Never commit your .env file — add it to .gitignore to prevent accidentally pushing the key to a public repository.

Activity 1.2: Research and Select the Appropriate Generative AI Model

- **Understand the Project Requirements:** Review the specific needs of DebtEase AI, focusing on generating empathetic, financially accurate negotiation letters.
- **Explore Gemini's Model Documentation:** Examine available Gemini models, including context windows, speed, and safety settings.
- **Evaluate Model Performance:** Compare models based on response quality, latency, and reliability for financial/legal-adjacent text generation.
- **Select the Optimal Model:** Choose Gemini's general-purpose text model with a moderate temperature (~0.6) for a balance of consistency and natural, empathetic tone.

Activity 1.3: Define the Architecture of the Application

- **Draft an Architectural Diagram:** Visual representation including the frontend (React.js), FastAPI backend, and Gemini API integration.
- **Detail Frontend Functionality:** Outline how borrowers enter loan details, view their dashboard, and request settlement predictions or negotiation letters.
- **Outline Backend Responsibilities:** Specify how FastAPI handles incoming requests to `/api/negotiation/generate-letter`, validates input, builds a structured prompt, and communicates with the Gemini API.
- **Describe AI Integration Points:** Define how the backend constructs prompts using borrower financial data and tone guidelines, calls the Gemini API, parses the JSON response, and returns structured results to the frontend.

Activity 1.4: Set Up the Development Environment

- **Install Python and Node:** Ensure Python 3.11+ and Node.js 18+ are installed.
- **Create a Virtual Environment:** Isolate backend dependencies using venv:

```
D:\projects> python -m venv venv
D:\projects> venv\Scripts\activate
(venv) D:\projects> pip install -r requirements.txt
```

- **Install FastAPI and Uvicorn:**

```
(venv) D:\projects> pip install fastapi uvicorn[standard]
```

- **Install the Gemini SDK and python-dotenv:**

```
(venv) D:\projects> pip install google-generativeai python-dotenv
```

- **Set Up Application Structure:** Create folders for routers, models, schemas, services, and the React frontend (as detailed in the Project Executable Files document).

Milestone 2: Core Functionalities Development

Activity 2.1: Develop Core Content Generation Features

Implement the three core generation features — Financial Health Analysis, Settlement Prediction, and Negotiation Letter Generation — as coordinated services driven by structured borrower data.

- **Financial Health Engine:** Calculates EMI ratio, monthly surplus, and a 0–100 debt stress score from loan and income data.
- **Settlement Prediction Engine:** Recommends a settlement amount and confidence rationale, using AI with rule-based fallback.
- **Negotiation Letter Generator:** Returns a complete, lender-specific letter calibrated to the borrower's financial profile and chosen tone.

Activity 2.2: Implement the FastAPI Backend

Define Routes in FastAPI:

- Set up routers for authentication, loan management, financial analysis, and negotiation endpoints.

Process User Input:

- Accept borrower loan/income data via Pydantic request models on the loan management endpoints.
- Validate required fields (outstanding amount, EMI, overdue duration, income) before processing.

Integrate API Calls:

- Within the `/api/negotiation/generate-letter` route, call `build_prompt()` to construct a structured prompt from financial data and tone.
- Submit the prompt to the Gemini API using the `google-generativeai` SDK.
- Parse and validate the JSON response, with a fallback to rule-based templates if parsing fails.

Milestone 3: main.py Development

Milestone 3 focuses on creating the core logic of the FastAPI application, which serves as the backbone of DebtEase AI. In this milestone, each feature's functionality is set up through FastAPI routes, reliable prompt construction and Gemini API interaction are established, and unit tests confirm each feature performs as expected.

Activity 3.1: Writing the Core Application Logic

Home / health-check route:

```
@app.get("/")
def read_root():
    return {"status": "DebtEase AI backend is running"}
```

Negotiation letter generation endpoint:

```
@app.post("/api/negotiation/generate-letter")
def generate_letter(request: NegotiationRequest, user=Depends(get_current_user)):
    if not request.loan_id:
        raise HTTPException(status_code=400, detail="loan_id is required.")

    try:
        prompt = build_prompt(request.financial_profile, request.tone)
        response = gemini_client.generate_content(
            prompt,
            generation_config={"temperature": 0.6, "max_output_tokens": 1024},
        )
        result = extract_json(response.text)

        if "letter" not in result:
            raise ValueError("Invalid response structure from AI")

        save_negotiation_history(user.id, request.loan_id, result["letter"])
```

```

        return {"success": True, "letter": result["letter"], "tone": request.tone}

    except ValueError:
        # Rule-based fallback if the AI response cannot be parsed
        fallback_letter = generate_fallback_letter(request.financial_profile, request.tone)
        return {"success": True, "letter": fallback_letter, "tone": request.tone, "fallback": True}
    except Exception as e:
        raise HTTPException(status_code=500, detail=f"Generation failed: {str(e)}")

```

Define Prompt Engineering Helpers:

TONE_DESCRIPTIONS dictionary: maps formal, empathetic, and firm tones to detailed copywriting instructions.

```

TONE_DESCRIPTIONS = {
    "formal": "professional and respectful, citing account details clearly and concisely.",
    "empathetic": "warm and understanding, acknowledging financial hardship while staying solution-
focused.",
    "firm": "direct and confident, clearly stating the proposed settlement terms.",
}

```

build_prompt(): assembles a structured prompt from the borrower's financial profile and tone, instructing the model to return strictly valid JSON.

```

def build_prompt(profile: dict, tone: str) -> str:
    tone_desc = TONE_DESCRIPTIONS.get(tone, "professional")
    return f"""You are an expert financial negotiation writer.
Write a {tone_desc} settlement negotiation letter using this borrower profile: {profile}.
Respond ONLY with valid JSON in the form {{"letter": "..."}}."""

```

Milestone 4: Frontend Development

Milestone 4 focuses on developing a clear, reassuring interface for DebtEase AI. This involves designing a clean, accessible layout with responsive React components, and creating dynamic content rendering to display AI-generated settlement recommendations and negotiation letters based on borrower interactions.

Activity 4.1: Designing and Developing the User Interface

Set Up the Base App Structure:

- Develop the main App.jsx as the routed application shell, including a header with the DebtEase AI logo, a navigation sidebar, and page routes for Login, Dashboard, Settlement Predictor, and Letter Generator.
- Use semantic HTML and accessible form elements throughout, since the primary audience may be under financial stress.

Design a Responsive Layout:

- Use Tailwind-style utility classes and CSS flexbox/grid to organize the dashboard into clear cards (financial health, active loans, recent negotiations).
- Add responsive breakpoints so the dashboard adapts across desktop, tablet, and mobile screens.

Create Separate UI Components:

- **Loan Cards:** individual cards per loan showing EMI, overdue duration, and a quick settlement-request action.
- **Financial Health Gauge:** a visual chart (via Recharts) showing debt stress score and EMI ratio.
- **Negotiation Letter Panel:** displays the generated letter with copy-to-clipboard and PDF download actions.

Activity 4.2: Creating Dynamic Content Rendering

Handle Form Submission Asynchronously:

- Attach an onSubmit handler to the loan/negotiation forms that sends data as a JSON POST request via Axios.
- During loading, disable the submit button and show an animated spinner in its place.

Render Results Dynamically:

- Update component state with the API response and conditionally render the settlement recommendation or negotiation letter panel.
- After rendering, smooth-scroll to the results section.

Restore UI State:

- In a finally block, re-enable the submit button and clear the loading state regardless of success or failure.

Milestone 5: Deployment

In Milestone 5, the focus is on deploying DebtEase AI first locally to verify correct operation, and then via Docker to a cloud host for evaluator/demo access. This involves configuring the server environment, managing dependencies, and launching the app for real-world use.

Activity 5.1: Preparing the Application for Local Deployment

Create and activate a virtual environment, then install all backend dependencies:

```
D:\projects> python -m venv venv
D:\projects> venv\Scripts\activate
(venv) D:\projects> pip install -r requirements.txt
```

Configure environment variables by creating a .env file in the project root:

```
GEMINI_API_KEY=your_gemini_api_key_here
JWT_SECRET_KEY=your_jwt_secret_here
```

Activity 5.2: Local Testing and Verification

Start the FastAPI development server:

```
(venv) D:\projects> uvicorn app.main:app --reload --port 8000
```

Once running, open a browser at <http://127.0.0.1:8000/docs> to access the interactive Swagger API documentation, and start the React frontend separately with `npm run dev`.

Interact with the settlement predictor and letter generator — test all three tones and multiple loan profiles — to verify AI responses are correctly generated, parsed, and rendered.

Activity 5.3: Containerized Deployment via Docker

Docker packages the backend and frontend into portable containers for consistent deployment across environments.

Build and run the containers using Docker Compose:

```
D:\projects> docker compose build
D:\projects> docker compose up -d
```

The compose file starts both the FastAPI backend (port 8000) and the React frontend (port 5173), connected on a shared network.

Important Notes:

- Use environment-specific .env files for staging vs. production so API keys are never hardcoded.
- Configure CORS in FastAPI to allow only the deployed frontend origin in production.
- Set up a managed database (e.g., PostgreSQL) before scaling beyond the development SQLite setup.

Exploring the Website's Web Pages:

Login / Registration Page

[Screenshot placeholder — insert once the UI is built] Description: A clean, centered card with email/password fields, a “Remember me” checkbox, and a prominent “Log In” button. A link below offers new borrowers a path to registration. The design uses calm, reassuring colors (soft blues/greens) appropriate for a financially stressed audience.

Dashboard / Financial Health Page

[Screenshot placeholder — insert once the UI is built] Description: The main landing page after login, showing a financial health gauge (debt stress score), key metrics (EMI ratio, monthly surplus), a list of active loans as cards, and quick-action buttons to request a settlement or generate a negotiation letter.

Settlement Predictor Page

[Screenshot placeholder — insert once the UI is built] Description: A form where the borrower selects a loan and requests a settlement recommendation. Results appear in a results panel showing the recommended amount, a plain-language rationale, and a button to proceed to the negotiation letter generator.

AI Negotiation Letter Generator Page

[Screenshot placeholder — insert once the UI is built] Description: A two-column layout — the left side shows the borrower's financial summary and a tone selector (Formal, Empathetic, Firm); the right side displays the AI-generated letter with copy-to-clipboard and “Download as PDF” actions.

Conclusion:

DebtEase AI is a generative AI platform built with FastAPI and React.js that streamlines debt management and settlement negotiation for financially distressed borrowers. It uses Google's Gemini API to instantly generate tailored settlement recommendations and negotiation letters, with a rule-based fallback ensuring the platform remains reliable even when the AI service is slow or unavailable. The project, which included model selection, development, and containerized deployment, highlights how targeted AI and a structured framework can meaningfully reduce the stress and complexity of debt recovery. Looking ahead, DebtEase AI offers significant opportunities for expansion, such as automated lender response tracking, multi-language negotiation letters, and a mobile app, positioning the platform to evolve from a specialized tool into a comprehensive financial recovery assistant.